Université Paris Cité

Compléments en Programmation Orientée Objet
Projet : Colony Flow

Projet à réaliser en binôme, à rendre sur Moodle, avant la date indiquée sur Moodle.
Le langage doit être Java (version 25).
Le code doit être écrit par vous, mais votre programme peut dépendre de bibliothèques tierces (celles indiquées dans le sujet ou des bibliothèques rendant un service équivalent).

1 À propos

Le projet consiste à concevoir et programmer en Java un jeu de réflexion en 2D, basé sur le même principe que Colony Flow (jeu sur Android, mais aussi dans le navigateur web : `https://colonyflow.games/`). Le joueur choisit des colonnes de cases contenant des insectes mangeurs de pixels afin de faire disparaître progressivement une image en pixel art.
Vous réaliserez votre propre jeu : les niveaux et graphismes pourront être créés par vous (et dans tous les cas judicieusement choisis pour illustrer les différents cas de figure).

2 Déroulement du jeu

Ce que voit le joueur La zone de jeu comprend trois parties :

1. Dans la moitié supérieure de l'écran, une image en pixel art, composée de pixels de plusieurs couleurs, que les insectes doivent manger.

2. Juste sous l'image, une rangée de cinq cases, vide au début de la partie. Ces cases servent de zone de lancement des insectes.

3. En bas de l'écran, une grille de trois rangées et un certain nombre de colonnes. Chaque case contient une boîte colorée d'une couleur esistant parmi les pixels de l'image, étiquetée par un nombre, qui est le nombre d'insectes dans la boîte.

Un trou, situé au milieu de l'écran, sert de point de retour aux insectes. Le joueur ne déplace pas les insectes directement : il choisit une colonne de la grille inférieure en cliquant dessus. Les boîtes de la colonne se décalent d'une case vers le haut et la boîte supérieure est placée dans une case libre de la rangée de lancement. Si cette rangée est pleine, aucune nouvelle boîte ne peut y être ajoutée.
Quant à la case libérée du bas de la colonne, elle est remplie par la boîte suivante (qui était cachée jusqu'ici).



FIG. 1 – Capture d'écran du jeu Colony Flow.

Les cases de la rangée de lancement libèrent les insectes qu'elles contiennent. Chaque insecte part de la boîte, mange un pixel accessible de la couleur correspondante dans l'image, puis revient disparaître dans le trou. Une case ne peut libérer ses insectes que tant qu'il reste dans l'image des pixels accessibles de sa couleur. Une fois son contenu traité, elle libère sa place dans la rangée. La partie est gagnée lorsque tous les pixels de l'image ont été mangés.

La totalité du modèle et les algorithmes En réalité il y a plus de boîtes par colonne que de rangées. Donc seules les premières boîtes de chaque colonne sont initialement connues par le joueur. La séquence des boîtes, avec la couleur et le nombre d'insectes de chacune est connue au départ pour chaque colonne. La contrainte est que le casse-tête doit être résolvable (le total des insectes pour chaque couleur correspond exactement au nombre de pixels de cette couleur dans l'image ; et il existe un ordre de sélection des boîtes qui permet ne ne pas être coincé).
Par ailleurs, cela suppose que les insectes se dirigent en priorité vers les pixels accessibles quit permettront de rendre au plus vite accessibles les couleurs qui correspondent aux boîtes en présence. Un algorithme :

- Repérer les boîtes déjà actives qui n'ont pas de pixels accessibles correspondants. Si elles ont toutes des pixels accessibles correspondants, on peut choisir une boîte de la rangée suivante, puis ainsi de suite.
- Repérer les pixels des couleurs correspondantes.
- Effectuer un parcours en profondeur depuis ces pixels pour trouver les pixels accessibles des couleurs déjà consommables qu'il faudra manger en premier.

Remarque : cet algorithme n'est qu'une heuristique qui favorise la terminaison du niveau. Il ne garantit pas totalement l'absence de blocage.
Pour faire fonctionner cet algorithme, il faut savoir aussi trouver les pixels accessibles. Pour cela, on peut utiliser un algorithme classique de recherche de chemin (comme A*) depuis les cases de la rangée de lancement. On pourra mettre en cache le résultat pour le déplacement effectif des insectes une fois qu'on a choisi les pixels qu'il faut consommer.
Enfin, chaque case de lancement vit sa vie indépendamment des autres. Dès que l'insecte qu'elle a envoyé a fini sa mission, elle choisit un pixel à consommer et lance un autre insecte (s'il en reste dans la boîte !) sans se demander ce que font les autres boîtes. La seule vérification est de ne pas cibler un pixel déjà ciblé par un autre insecte. La boîte se retrouve bloquée s'il n'y a plus de pixel accessible de la bonne couleur. Elle se débloquera toute seule dès que ce sera à nouveau le cas.
Pour les deux algorithmes, on considère que deux pixels sont voisins s'ils partagent un côté (haut, bas, gauche, droite). On ne considère pas les diagonales. Pour l'accessibilité, on considère qu'un chemin ne peut passer que par des pixels en dehors de l'image ou par des pixels déjà mangés (vides).

3 À programmer

La consigne est simple : en utilisant un maximum de principes de POO vus en cours cette année, programmer ce jeu de réflexion où des insectes mangent une image en pixel art.
Les fonctionnalités à implémenter :

La base :

1. Modélisation d'un niveau : image en pixels, couleurs, colonnes de cases et quantité d'insectes dans chacune. Vérifiez la validité des données et la cohérence entre les couleurs des cases et celles de l'image.

2. Implémentation des règles de consommation : sélection d'une colonne, transfert de sa case supérieure vers une case de lancement libre, départ des insectes, choix d'un pixel accessible de la bonne couleur (quel que soit l'algorithme utilisé) et mise à jour de l'image et des quantités.

3. Définition de l'accessibilité des pixels et de la manière dont les insectes sont traités lorsqu'une case de lancement ne peut plus envoyer d'insectes.

4. Détection de la victoire lorsque l'image a été entièrement mangée, et indication de la progression (par exemple, pixels restants).

5. Inversement, détection de l'échec lorsque la rangée de lancement est pleine et qu'aucune case ne peut plus envoyer d'insectes.

6. Chargement des niveaux depuis un fichier, dans un format de stockage lisible et documenté.

7. Interface graphique représentant les trois parties de la zone de jeu et le trou central. Les colonnes sont sélectionnées à la souris ou au clavier. Une animation des insectes est facultative.

8. Menu de démarrage, sélection ou enchaînement de plusieurs niveaux, et possibilité de recommencer un niveau.

Les fonctionnalités supplémentaires :

1. Les différents bonus du jeu Colony Flow (ou d'un de ses nombreux clones) : mélanger les boîtes, aller chercher une boîte en dessous, supprimer tous les pixels d'une couleur choisie, etc.

2. Algorithme de choix de pixels accessibles à consommer favorisant de ne pas se retrouver bloqué (celui décrit dans l'énoncé ou un autre), voire garantissant cela (si vous savez le prouver...)

3. Multithreading pour le traitement des d'insectes, avec synchronisation correcte de l'accès à l'état du jeu (un *thread* par boîte de départ). Penser à gérer le blocage et le redémarrage des boîtes bloquées à l'aide de la synchronisation vue en cours.

4. Génération de niveaux réalisables, depuis une image donnée. Il faut donc générer une séquence de boîtes pour chaque colonne, avec des couleurs et des quantités d'insectes cohérentes avec l'image.

4 Contraintes techniques

Architecture Séparez la représentation du jeu et ses règles de l'interface graphique. Le modèle doit pouvoir être utilisé et testé sans lancer l'interface (notamment par les tests unitaires). Les niveaux ne doivent pas être codés en dur dans les composants graphiques : choisissez un format de données adapté et documentez-le.
Le multithreading n'est pas nécessaire pour le jeu de base. Si vous l'utilisez, veillez à synchroniser correctement les accès à l'état du jeu et à respecter les contraintes du toolkit graphique.

Testabilité Tout le code que vous écrirez devra avoir été programmé de telle sorte à ce que le comportement soit testable.
Le découpage en méthodes doit être suffisamment fin pour permettre des tests unitaires pertinents. Testez notamment le décalage d'une colonne, le refus d'un choix lorsque la rangée de lancement est pleine, le transfert des cases, la sélection des pixels accessibles de la bonne couleur, la mise à jour des quantités d'insectes et les conditions de victoire et de défaite.

Le découpage en classes et la programmation « à l'interface » doivent permettre d'effectuer facilement des tests d'intégration. Cela peut être fait en remplaçant n'importe quel composant par un composant factice implémentant la même interface.
Les tests devront être écrits, exécutables, et fournis avec le rendu.

5 Aide technique

- Utilisez un système de compilation tel que Maven ou Gradle. Configurez le projet pour Java 25 et facilitez la compilation, l'exécution et le lancement des tests.
- Interface graphique : Swing, JavaFX[1] (vous pouvez aussi consulter les transparents du cours sur les IG, qui sont basés sur JavaFX) ou autre. Pour tout toolkit autre que Swing, l'installation des dépendances doit se faire automatiquement avec Maven ou Gradle.
- Les tests peuvent être écrits dans un *framework* tel que JUnit5[2], mais ce n'est pas indispensable. En tout cas, il faut des tests.

6 Critères d'évaluation

- Le projet est rendu sur GitLab (moule). La dernière version du projet disponible sur la branche par défaut à la date de rendu sera considéré comme votre rendu.
- **Pour que votre projet GitLab soit connu de nous, il est important de le créer en tant que bifurctation *fork*) du projet de base (l'adresse du projet de base sera donnée sur Moodle). Par ailleurs vous nommerez votre projet "`projet-cpoo5-2627-session1-nom1-nom2`", où `nom1` et `nom2` sont vos noms de famille, en ordre alphabétique.**
- Le projet doit contenir un fichier `README.md` indiquant comment compiler, exécuter et utiliser votre programme et ses tests, décrivant les fonctionnalités effectivement implémentées et expliquant vos choix techniques originaux, le cas échéant.
- Joignez aussi un ou des diagrammes de classe.
- La commande pour compiler ou exécuter doit être simple (s'aider de Gradle, Maven... voire de Makefile).
- La compilation selon ces instructions doit terminer sans erreur ni avertissement.
- L'exécution doit être correcte : elle respecte le cahier des charges et ne quitte pas de façon non contrôlée.

  Notamment, en mode graphique, les exceptions doivent être rattrapées. Dans tous les cas, si le problème n'est pas réparable, un message d'erreur doit être présenté à l'utilisateur. Aucune de ces erreurs ne doit être due à un problème de programmation (comme le fameux `NullPointerException`...).
- Le code respecte les conventions de codage.
- Le code est architecturé intelligemment. Notamment, il faut utiliser des patrons de conception vus en cours ainsi que les constructions les plus appropriées fournies par Java.
- Les classes et méthodes sont documentées (javadoc).

[1] `https://openjfx.io/`
[2] `https://junit.org/junit5/`

- Des commentaires expliquent les portions de codes qui ne sont pas évidentes.
- Les tests fournis doivent être aussi exhaustifs que possible.
- Un projet qui tenterait de tout faire mais dans lequel rien ne marche bien sera moins bien noté qu'un projet qui ne remplit que la première moitié des objectifs sans avoir de bug.

  Les objectifs sont incrémentaux. Il est toujours possible de rendre un projet cohérent qui n'aurait atteint que les premiers objectifs, donc profitez-en !

À propos de l'usage de l'IA Vous devez être capable d'expliquer et de prendre la responsabilité de votre code et de vos choix techniques devant le jury, qui vous posera de nombreuses questions. La consigne est que le projet doit être fait par vous-même. Si toutefois un morceau est fait à l'aide d'un assistant IA, on vous demande de le signaler clairement (vous serez pénalisés durement si, suite à nos questions, on comprend que vous avez menti). Seules les parties codées à la main vous rapporteront des points.
L'entretien servira à prouver que

- Vous savez ce que vous faites et de nous convaincre que ça correspond aux principes de programmation vus en cours.
- Vous savez programmer sans l'IA (on vous demandera probablement de programmer une fonction devant nous pendant la soutenance).